Associate Software Engineer Development Task

Fawry Payment Routing Engine

Task Duration: 7 Days (+1 Day optional for bonus)

Introduction

Dear Candidate,

Congratulations on being selected to participate in the development task for the Associate Software Engineer position at Fawry. This document outlines a comprehensive technical assessment designed to evaluate your problem-solving abilities, code quality, and understanding of payment systems architecture.

This task simulates a real-world scenario that Fawry encounters daily: intelligent payment routing across multiple payment gateways to optimize cost, speed, and reliability for our billers.

Task Overview

Scenario: Fawry processes payments through multiple payment gateways (Ex: Vodafone Cash, Etisalat Cash, Bank Transfer, Credit Card). Each gateway has different characteristics including processing times, commission rates, daily limits, and availability windows.

Your Mission: Build a smart payment routing system that recommends the best gateway for a given payment, handles partial transactions when limits are reached, tracks commission calculations, and provides dashboard visibility through both backend APIs and an Angular frontend.

Sample Gateway Configurations

Gateway	Gateway 1	Gateway 2	Gateway 3
Commission	2 EGP + 1.5%	5 EGP + 0.8%	0 EGP + 2.5%
Daily Limit	50,000 EGP	200,000 EGP	100,000 EGP
Processing Time	Instant	24 Hours	2 Hours
Availability	24/7	Sun-Thu 9AM-5PM	24/7
Min Transaction	10 EGP	100 EGP	50 EGP
Max Transaction	5,000 EGP	No Limit	10,000 EGP

Core Requirements

1. Gateway Service

Design a flexible system to store and manage gateway configurations. Your solution should allow adding or modifying gateway rules without code changes.

- Store gateway configurations
- Support dynamic gateway addition and configuration updates
- Calculate accurate commission for any gateway and amount combination
- Validate transaction amounts against gateway limits

2. Smart Routing Algorithm

Implement an intelligent routing algorithm that recommends the optimal gateway based on multiple factors.

Routing Factors:

- Amount (must fit within gateway transaction limits)
- Urgency (INSTANT, CAN_WAIT)
- Cost optimization (minimize commission)
- Gateway availability (check if operational at request time)
- Daily quota remaining (ensure biller hasn't hit their limit - quota is tracked per biller and resets at midnight)

API Endpoint Example:

```
POST /api/payments/recommend

Request:
{
  "billerId": "BILL_12345",
  "amount": 1000,
  "urgency": "INSTANT"
}

Response:
{
  "recommendedGateway": {
    "id": "gateway_1",
    "name": "Gateway 1",
    "estimatedCommission": 17.0,
    "processingTime": "Instant"
  },
  "alternatives": [
    { "id": "gateway_3", "estimatedCommission": 25.0 },
    { "id": "gateway_2", "estimatedCommission": 13.0 }
  ]
}
```

Note: alternatives[] should contain all other viable gateways ranked by your scoring algorithm.

3. Transaction History Log

Create backend APIs that provide billers with visibility into their transactions history. This log should display transactions processed by the routing engine, showing which gateway was selected and the final status.

Build an Angular component that displays transaction history with filtering by date and gateway. The UI should clearly present the data returned by your API.

API Endpoint Example:

```
GET /api/billers/{billerId}/transactions?date=2025-01-15
```

Response should include:

- Total amount processed for the day
- Total commission charged
- Breakdown by gateway with transaction count, amounts, and quota usage

Bonus Challenge. Payment Splitting Logic

Handle payments that exceed individual gateway transaction limits by intelligently splitting them across the same gateway.

Example Scenario: A biller needs to process a payout of **12,000 EGP**, but when your routing algorithm selects the optimal gateway (e.g., Vodafone Cash with a **5,000 EGP** max limit), the amount exceeds the limit. Your system should automatically detect this and split the payment.

Your Task: Extend your routing algorithm to handle "over-limit" payments by splitting them into multiple transactions using the recommended gateway.

Your algorithm should:

- **First, select the optimal gateway** using your existing routing logic based on the urgency and amount.
- **Detect if splitting is needed:** Check if the payment exceeds the gateway's max transaction limit.
- **Calculate Split Amounts:** Break down the total amount into chunks that respect the Max Transaction limit (e.g., 12,000 EGP → 5,000 + 5,000 + 2,000).
- **Validate Minimum Limits:** Ensure each split transaction meets the gateway's minimum transaction requirement.
- **Commission Calculation:** Calculate the total commission for all split transactions combined.
- **Quota Validation:** Check if the biller has enough remaining daily quota on that gateway to process all splits.

API Endpoint Example:

POST /api/payments/split

Request: { billerId, amount, urgency }

Response: { selectedGateway: "Gateway1", requiresSplitting: true, splits: [5000, 5000, 2000], totalCommission: 156.50, quotaAvailable: true, splitCount: 3 }

Deliverables

Please ensure your submission includes all of the following:

1. **Spring Boot REST API** - Fully functional backend with all required endpoints
2. **Angular UI** – Fully integrated with your developed backend
3. **Database Setup**
4. **README Documentation:** README with setup instructions and architecture explanation
5. **Postman Collection** - Exported JSON file with all API endpoints and sample requests

Technical Requirements

- **Language & Framework:** Java 17 with Spring Boot 3.x
- **Angular (16+)**
- **Database:** Oracle or PostgreSQL
- **Authentication:** JWT
- **Version Control:** Git

Timeline

Deadline: 7 days from receipt of this document or 8 days if you decide to implement the bonus

Submission Instructions

1. Push your code to a GitHub repository (public)
2. Ensure your README contains clear setup instructions
3. Include your Postman collection in the repository
4. Send the repository link via email to the recruiter

Follow-up Code Review Session

After submission, selected candidates will participate in a 30-minute live code review session where you will:

- Walk through your solution and explain key design decisions
- Discuss trade-offs and alternative approaches you considered
- Answer technical questions about your implementation
- Potentially make a small live modification to your code

Important Notes

- **This is NOT a copy-paste exercise.** We're interested in YOUR thinking and problem-solving approach, not just working code.
- **Quality over quantity.** A well-implemented subset of features with excellent code quality is better than a complete implementation with poor code.
- **Use of AI tools is allowed** but you must understand and be able to explain every line of code. The code review session will quickly reveal if you understand your own solution.
- **Focus on core functionality first** before adding extras. A solid implementation of requirements 1-3 is better than a rushed implementation of all 4.
- **Ask questions if needed.** If any requirements are unclear, reach out to your recruiter. Clear communication is part of what we're evaluating.

Good luck with the task! We look forward to reviewing your solution.

Fawry Payouts Team